

STL контейнери - 1. управляват динамична памет

2. спазват RAII

3. имат правилни copy/move семантики

4. освобождават ресурсите автоматично

5. няма нужда от delete

`std::string`

`::vector`

`::array`

`::map`

`::unique_ptr`

`::shared_ptr`

`std::array`

- има фиксиран размер (по време на компилация)
- няма динамично разширяване
- няма допълнителна памет
- използва се при: малък, фиксиран размер

`std::vector`

- има динамичен размер (run-time)
- има динамично разширяване
- има допълнителна памет (за ~~capacity~~ capacity)
- използва се: много често

`std::list` - често се махат и добавят елементи в средата
- пазят се валидни итератори

Масиви от указатели - при създаване викаме new, но след това нямаме информация кой е отговорен за delete.

Решението: smart pointers

Проблема: масиви от raw pointers (сурови указатели) + move

Ръчно управление на памет води до:

- memory leaks, double delete ...

работи добре с
типове, които
моделират ownership

Smart pointer - обект, който се грижи (управлява) живота на друг обект и автоматично освобождава ресурса в деструктора си

std::unique_ptr - единствен собственик (само 1 обект притежава ресурса)
• не се копира (^{copy}=delete), само се премества (move е позволен)
• избираме го по подразбиране

std::shared_ptr - един обект може да има няколко собственика
• използва reference-counting, при =0 (броята) обектът се трие
• възможно е да се случи цикъл и броята никога да не е 0

std::weak_ptr
• не увеличава reference count
• използва се за избягване на цикли

Добра практика: - да се мисли за ownership още при дизайна
- да се използва make_unique/make_shared

Rule of Zero: при класове, които правилно управляват ресурсите си,
нема нужда от специални функции (компилатора ги генерира)